

Feedback-Generation for Programming Exercises With GPT-4

Imen Azaiz
imen.azaiz@ifi.lmu.de
LMU Munich
Munich, Germany

Natalie Kiesler
natalie.kiesler@th-nuernberg.de
Nuremberg Tech
Nuremberg, Germany

Sven Strickroth
sven.strickroth@ifi.lmu.de
LMU Munich
Munich, Germany

ABSTRACT

Ever since Large Language Models (LLMs) and related applications have become broadly available, several studies investigated their potential for assisting educators and supporting students in higher education. LLMs such as Codex, GPT-3.5, and GPT-4 have shown promising results in the context of large programming courses, where students can benefit from feedback and hints if provided timely and at scale. This paper explores the quality of GPT-4 Turbo's generated output for prompts containing both the programming task specification and a student's submission as input. Two assignments from an introductory programming course were selected, and GPT-4 was asked to generate feedback for 55 randomly chosen, authentic student programming submissions. The output was qualitatively analyzed regarding correctness, personalization, fault localization, and other features identified in the material. Compared to prior work and analyses of GPT-3.5, GPT-4 Turbo shows notable improvements. For example, the output is more structured and consistent. GPT-4 Turbo can also accurately identify invalid casing in student programs' output. In some cases, the feedback also includes the output of the student program. At the same time, inconsistent feedback was noted such as stating that the submission is correct but an error needs to be fixed. The present work increases our understanding of LLMs' potential, limitations, and how to integrate them into e-assessment systems, pedagogical scenarios, and instructing students who are using applications based on GPT-4.

CCS CONCEPTS

• **Social and professional topics** → **Student assessment**; • **Computing methodologies** → **Artificial intelligence**.

KEYWORDS

formative feedback, personalized feedback, assessment, introductory programming, Large Language Models, LLMs, GPT-4 Turbo, benchmarking

ACM Reference Format:

Imen Azaiz, Natalie Kiesler, and Sven Strickroth. 2024. Feedback-Generation for Programming Exercises With GPT-4. In *Proceedings of the 2024 Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*, July 8–10, 2024, Milan, Italy. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3649217.3653594>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ITiCSE 2024, July 8–10, 2024, Milan, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0600-4/24/07

<https://doi.org/10.1145/3649217.3653594>

1 INTRODUCTION

Large Language Models (LLMs) not only took the world by storm, they also have a potentially great impact on programming education comprising both opportunities and challenges for learners and educators [30]. With the release of OpenAI's new model GPT-4 Turbo in November 2023, an updated knowledge cutoff until April 2023, and an increased context window were made available along with the extension to become a multimodal model [28]. The new model yet again raises the question of to what extent generative AI tools can be used to create truly individual, reliable feedback that is adequate for novice learners of programming.

Individual feedback may help counteract the challenges well-known in the context of introductory programming classes and improve student performance, if addressing students' (informational) needs [11, 27, 34]. Novice learners of programming usually face several challenges in the introductory phase of their studies. Programming may be completely new to them due to their educational biography, and it is considered a cognitively complex tasks [15, 16], involving cognitive challenges (e. g., problem understanding, developing algorithms, debugging, understanding error messages [7, 8, 24, 35]). Moreover, expectations from educators and institutions towards students seem to be too high and unrealistic [23, 24, 42].

At the same time, educators struggle with high student numbers, limited resources to provide feedback and hints, lack of tutors, and an overall heterogeneity of their students [29, 37]. Considering the scenarios of (ungraded) formative assessments to support students' learning process, receiving feedback is key for them to improve their work during the semester. It is therefore not surprising that many (formative) e-assessment systems have been developed to support both educators and students [12, 14, 40]. Yet, many of the current systems only focus on functioning code and automatic tests instead of individualized feedback.

In this paper, **the goal** is to explore the capabilities of GPT-4 Turbo to generate formative feedback for programming exercises. The research question (RQ) is *How can we characterize the feedback provided by GPT-4 Turbo if provided with a task description and a student solution as input?* By answering this RQ, we contribute to the body of research improving the computing education research community's understanding of LLMs and their potential benefits for novice programmers. We also discuss the use case of applying the GPT-4 API as part of a university's e-assessment system.

Therefore, this work has implications for educators and e-assessment system developers considering the integration of the GPT-4 API into their courses or systems. As ChatGPT is based on GPT-4, this research also has implications for students seeking help for a certain issue, and educators instructing students on the conscious and critical use of GPT-4's feedback.

2 RELATED RESEARCH

In the past decades, numerous e-assessment systems, intelligent tutoring systems (ITS), and learning environments have been developed to provide automatic feedback and hints to students [12, 14, 40]. These systems can provide timely feedback at scale without the need for an educator’s intervention. For most systems, test cases are automatically executed upon a student’s submission to generate feedback [14]. A precondition for this to work is the development of such tests (plus a domain model in case of an ITS), causing tremendous effort for educators. Some systems also employ professional code analysis tools such as PMS, CheckStyle, or SonarCube to provide feedback on style (e. g., [22]). A common problem is, however, that the provided feedback is not always useful for students, as the descriptions lack details on how to proceed [3, 13].

In the context of ITS, using AI has a long-standing tradition [19]. More recently, LLMs have become widely available and are being explored for application as feedback generators for novice learners of programming. Several papers including a recent ITiCSE working group report [2, 6, 30] discuss implications of recent AI advances for computing education, particularly for introductory programming courses. Early papers used OpenAI’s Codex model and demonstrated its ability to solve CS1 and CS2 programming tasks at a level similar to students. Occasionally, it faced difficulties with output formatting, odd edge cases, ambiguous requirements, and wordy tasks [9, 10].

LLMs have also shown to be able to create effective code explanations as well as enhance programming error messages [21, 25, 32]. MacNeil et al. conclude that the majority of students perceived the automatically generated line-by-line code explanations by an LLM as helpful when evaluated as part of an e-book [25]. Leinonen et al. conclude that code explanations generated by GPT-3 are rated better on average w.r.t. understanding and accuracy than explanations created by students. Moreover, students were not averse to feedback generated by LLMs, and they prefer line-by-line explanations [20].

Considering the context of LLM-generated feedback (GPT-4) to programming problems and how to elicit it, the accuracy of the feedback seems to improve when the model receives the task instruction as input [4]. At the same time, such input seems to cause a decreasing LLM performance in identifying errors, indicating the need for more research [4]. A study on the generation of next-step hints by GPT-3.5 [31] recommends the use of the task description and keywords as input. Regardless of the input, LLM-generated feedback messages may contain misleading information [17] and lack sufficient detail when students approach the end of the assignment [31]. Azaiz et al. note further difficulties of GPT-3.5 with output formatting, hallucinating errors, and recognizing correct solutions, resulting in adequate feedback in only 47 % of the cases.

Other recent studies on GPT-4 investigate its use to localize errors in program code [43], and to what extent it passes assessments from introductory and intermediate Python classes [33]. However, the latest version of OpenAI’s LLM (GPT-4 Turbo) [28] has not yet been subject to qualitative research regarding its feedback capabilities. Moreover, the breadth and depth to which we explore the feedback structure and quality in the context of introductory programming is a novel contribution to the computing education research community.

3 METHODOLOGY

The goal of this work is to explore the formative feedback generation capabilities of GPT-4 Turbo for introductory programming exercises. Our research is guided by the following RQ: *How can we characterize the feedback provided by GPT-4 Turbo if provided with a task description and a student solution as input?*

To evaluate the feedback generated by GPT-4, a qualitative empirical study was conducted. Two assignments from an introductory Java course at LMU Munich, a large German university, were selected along with authentic student data. The authors obtained and reused available student data from related work [1]. The dataset contains all submissions from 695 computer science students (majors and minors), who took a first-year introductory programming class in the winter term 2021/22. The course was accompanied by weekly homework assignments and peer reviews. Participation was voluntary. The e-assessment system GATE [39] was used to collect the submissions, to provide instant feedback for some tasks, and to facilitate the peer review process [36]. This research utilizes submissions from students who explicitly consented to their use (695 out of about 900 students). Consent was fully voluntary, with no negative consequences or disadvantages.

The first task we selected from the dataset required students to: “Write a Java application named *SimpleWhileLoop* that uses a *WHILE* loop to count and prints all odd numbers from 1 to 10, and then prints ‘Boom!’ (without quotation marks) afterward.” It was expected in week 2 of the class. The second task we selected was due in week 7. It is object-oriented and expects students to implement an interface, use an inner-class, write multiple methods, traverse linked lists, and manage references. The assignment specification is: “Implement the *Queue* interface according to the specification (in the interface) for a queue with the *QueueImpl* class by using a singly linked list.” The Java interface *Queue* was provided as a Java file containing the following five methods with JavaDoc specifying the semantics: *void append(int value)*, *boolean isEmpty()*, *void remove()* (null operation for an queue list), *int peek()* (*EMPTY_VALUE* of the interface should be returned on an empty queue), and *int[] toArray()*.

About 9 % of all submissions for these two assignments were used in this study: 33 submissions were pseudo-randomly sampled for the first assignment and 22 randomly for the second one. During our review of the selected solutions, we found that two submissions for the second task were very similar, i. e. only differing w.r.t. the extensive use of comments. We kept both due to their authenticity.

For the generation of feedback, we used the GPT-4 Turbo (gpt-4-1106-preview) model with default settings and the following prompt template, following the methodology in related work [1]:

[ASSIGNMENT INSTRUCTIONS]

Find all kinds of errors, including logical ones, and provide hints for their correction or improvement, including suggestions for code style.

[CODE OF STUDENT SUBMISSION]

We experimented with several variations of the prompt but could not find significant differences. Next, the feedback was generated three times for every submission in randomized order using the very same model and configuration (zero-shot approach). The submissions of the first assignment were processed on November, 21st 2023 resulting in 99 feedback texts. The submissions of the second task were processed on January, 4th 2024, resulting in 84 outputs.

All outputs were manually analyzed using a qualitative thematic analysis technique [5, 26]. The classification in related work [1] was used as a starting point for the deductive-inductive category-building process. Inductive categories were developed based on the material to describe new feedback characteristics. Three computing education researchers with extensive expertise in correction and providing feedback as well as qualitative analysis were involved in the coding and development of the categories.

In addition to the thematic analysis, all submissions were manually checked using unit tests for syntactic and functional correctness. Functional correctness assumes the submission fulfills the task specification and works as expected (i. e., regardless of its performance). Moreover, we evaluate GPT’s accuracy, precision, and recall. For all categories representing feedback characteristics, we counted the frequencies related to either of the tasks.

4 RESULTS

In this section, we present the results of the investigation of the feedback generated by GPT-4 Turbo. We characterized the feedback based on its form (e. g., content, structure, length, and overall composition), and evaluated the correctness of the feedback before examining the types of corrections provided by the model. Moreover, the results reflect code optimizations, style recommendations, inconsistencies, and redundancies. Variations of the feedback characteristics depending on the assignments are discussed related to these five main characteristics. Table 1 represents the codebook, whereas examples are provided in the text, where appropriate.

4.1 Feedback Content, Structure, and Length

The deductive-inductive characterization of the feedback generated by GPT-4 started with the development and application of categories reflecting its content and structure (see Table 1). We further analyzed the length of the responses.

4.1.1 Content and Structure. Regarding the content of the generated feedback, we found that 100 % of the output contained both code and text (FTWC) as shown in the first section of Table 2. Overall, we found the content of the generated output to be “individualized” to the input, meaning there were very few repetitive elements in all of the LLMs’ responses. The content was almost always compliant with the assignment (in 98 %). In only one response to a student solution for the *SimpleWhileLoop*, the odd numbers followed by the word “Boom” were not displayed line-by-line in any of the three iterations. Yet, GPT-4 Turbo acknowledged this issue in the textual output.

In general, the feedback generated by GPT-4 Turbo seemed to exhibit a certain structure, usually comprising sections. The first part is an introductory statement or a description of the submitted code. This is coupled with an assessment of the code’s quality and correctness. Next, an (enumerated) list of issues and respective corrections or suggestions for improvements were mostly displayed. These were accompanied by the improved version of the complete code (FuCo) or code snippet (CoSn). Usually, these list items were categorized under various labels such as “Logical Errors”, “Corrections and Improvements”, “Code Style and Clarity”, “Code Efficiency”, “Error Handling”, or “Variable Naming”. As a last part, GPT-4 Turbo generated a summary (of its corrections) along with final remarks.

Table 1: Coding book with descriptions (examples are provided in the text where appropriate)

Category	Description
<i>Feedback Content and Structure</i>	
Feedback without code (FWOC)	Feedback contains plain text without code (lacking Java programming language keywords or variable/method names).
Feedback text with code (FTWC)	Feedback contains text with code, snippet, variable/method name.
Feedback just containing code (FJCC)	Feedback contains only code.
Compliance with spec. (CWAS)	Corrections or suggestions align with the provided instructions and assignment specification.
<i>Code Representation</i>	
Full code (FuCo)	Suggests a full program sample solution.
Code snippet (CoSn)	Corrects small portions of the program suggesting a sequence of instructions.
Code snippet with instruction (CoSnI)	Generates code snippets with gaps, including instructions for students on how to fill in the remaining gaps.
Code with output (CWO)	Suggests improvements in the code with the corresponding output.
Inline code correction (ICC)	Feedback text contains student solution with inline comments (corrections and suggestions).
<i>Correctness and Correction Types</i>	
Only correct correction/suggestions (OCCS)	Feedback contains only correct improvements/suggestions, meaning all contained errors were fixed. Moreover, all of the suggestions have been implemented, resulting in the display of running code.
Partially correct correction/suggestion (PCCS)	Only some feedback components are correct, while other components introduce new issues (i. e., incorrect feedback or suggestions).
Only false correction/suggestion (OFCS)	Feedback contains only false corrections like non-existent errors or suggestions resulting in broken code.
Completely correct correction (CCC)	Feedback addresses all of the submitted code’s issues, contains only correct corrections, and adheres to the task requirements. Applying the feedback results in a fully correct submission.
(Fault) localization (FL)	Bugs are identified and localized, e. g., by citing code snippets, or describing them.
(Fault) localization correct (FLC)	Bugs are correctly identified and localized and are present in these locations.
<i>Suggested Optimizations and Coding Style</i>	
Optimization (OPT)	Suggests optimizations regarding the functionality of the program.
Code style suggestion (CSS)	Suggests improvements regarding readability, documentation, comments within the code, variable naming, etc.
Language suggestion (LCS)	Feedback contains translations and language related suggestions.
<i>Inconsistencies and Redundancies</i>	
Inconsistency (InC)	Recommendation does not correspond to the sample solution, or contradiction within the textual feedback.
Redundancy (RD)	Repeats the same suggestion in the same feedback or provides a suggestion that is already implemented in the code.

The order of hints and corrections, however, often seems random, e. g. coding style hints are provided before errors, recommendations for encapsulation are not bundled, or a missing inner class is provided as the last point. Another observation is the repetition of several text fragments across the responses. Those were, for example, related to code style: “Always use curly braces ‘{}’ for blocks under ‘if’ statements and loops”.

Table 2: Frequencies of all codes applied to both tasks

	SimpleWhileLoop n = 33			Queue n = 22			All n = 165	
Char.	1st	2nd	3rd	1st	2nd	3rd	Sum	%
<i>Feedback Content and Structure</i>								
FWOC	0	0	0	0	0	0	0	0
FTWC	33	33	33	22	22	22	165	100
FJCC	0	0	0	0	0	0	0	0
CWAS	32	32	32	22	22	22	162	98
<i>Code Representation</i>								
FuCo	33	33	33	12	10	11	132	80
CoSn	1	3	2	10	14	14	44	27
CoSnI	0	0	0	1	0	1	2	1
CWO	5	1	5	0	0	0	11	7
ICC	7	3	3	0	0	0	13	8
<i>Correctness and Correction Types</i>								
OCCS	21	23	19	13	10	13	99	60
PCCS	12	10	14	9	12	9	66	40
OFCS	0	0	0	0	0	0	0	0
CCC	21	19	18	11	8	9	86	52
FL	18	19	20	21	21	22	121	73
FLC	16	13	17	19	21	20	106	64
<i>Suggested Optimizations and Coding Style</i>								
OPT	19	10	5	19	21	22	96	58
CSS	33	33	33	19	21	20	159	96
LCS	6	7	6	5	4	2	30	18
<i>Inconsistencies and Redundancies</i>								
InC	6	6	7	6	4	8	37	22
RD	3	1	2	6	5	0	17	10

4.1.2 Length. In addition to the construction of inductive categories, we analyzed the length of the responses, as they were extensive. Table 3 shows the word counts for both assignments and across all three runs. The number of words was determined by tokenizing the feedback string using the white space (“\s+”) and counting the resulting tokens. The overall median feedback length is $m = 360$ words ($\bar{x} = 381$). It seems to be quite consistent across the three runs. The generated feedback for the *SimpleWhileLoop* assignment ($m = 312$) is shorter than for the *Queue* ($m = 470$). This difference is statistically significant (Mann-Whitney UTest, $U = 168$, $p < .001$, two-sided).

4.2 Code Representation

The representation of code was another theme we identified in the responses in varying forms. The second section of Table 2 highlights the variation of the feedback representing, for example, the full code (FuCo) and code snippets (CoSn) as suggestions. Specifically, every feedback for the *SimpleWhileLoop* included full code. Only three

Table 3: Length of the generated feedback by word counts (OA: over all iterations for each assignment)

	SimpleWhileLoop				Queue				
Words	1st	2nd	3rd	OA	1st	2nd	3rd	OA	All
Mean	309	310	324	315	466	504	477	482	382
Median	309	306	325	312	456.5	508	477.5	470	360
Min	219	223	168	168	357	409	339	339	168
Max	407	423	483	483	579	610	619	619	619

occurrences of code snippets were identified. In contrast to that, about half of the feedback generated for the *Queue* task contained full code. The other half only contained code snippets.

The other three characteristics related to the code’s representation also varied depending on the assignment. Code snippets with instructions (CoSnI) were provided exclusively in the *Queue* assignment’s feedback. This usually took the form of a code snippet with guidance for the student on how to continue. Inline code corrections (ICC), and code paired with its corresponding output (CWO) were only generated in response to the *SimpleWhileLoop*, appearing in 13 and 11 outputs respectively.

4.3 Feedback Correctness and Correction Types

Before characterizing the LLMs’ output correctness and their correction types, we briefly indicate the quality of the students’ code and their errors. The majority (57 %) of the student solutions for the *SimpleWhileLoop* is fully correct (with 90 % having syntactic correct), whereas 5 % of the *Queue* submissions are fully correct (with 59 % containing syntactic correct).

Evaluating GPT’s classification performance in finding errors was used as a starting point, before constructing inductive categories to describe the output’s correctness and types of corrections. When we started to explore the correctness of GPT’s feedback to students’ submissions, it was often not explicit whether GPT had classified a submission as correct or incorrect. In the absence of explicit judgments, we used terms such as “error” and “correction” to develop categories reflecting GPT’s corrections. It is important to note the difference between corrections and suggestions for improvement. We thus did not categorize suggestions regarding code style and optimization as errors.

Table 4 shows the results of the analysis w.r.t. GPT’s output, and its correctness. Overall, the accuracy (i. e., ratio of correct results to all results) ranges between .75 and .81 for the *SimpleWhileLoop* and between .9 and .95 for the *Queue*. The precision (i. e., ratio of correct positive results to all positive results) is optimal for the *Queue* and between .78 and .91 for the *SimpleWhileLoop*. However, the recall (i. e., ratio of correct positive results to all actual positive results) is better for the *SimpleWhileLoop* (.68). For the *Queue* task, it ranges between .33 and .66.

Most of the generated feedback texts contained correct corrections (OCCS, 60 %). No feedback contained false corrections only. However, in 40 % of the feedback the corrections were only partially correct (PCCS), meaning some of the feedback was incorrect (see Table 1). To localize errors, GPT-4 cites or highlights code, which we found in 73 % of the output (88 % of these errors were correct).

Table 4: Comparison of evaluation metrics of GPT-4 Turbo’s classification performance across the three runs for the two assignments

Metric	SimpleWhileLoop				Queue				All
	1st	2nd	3rd	OA	1st	2nd	3rd	OA	
Accuracy	.75	.81	.75	.77	.95	.90	.90	.91	.84
Precision	.78	.91	.78	.82	1.00	1.00	1.00	1.00	.91
Recall	.68	.68	.68	.68	.66	.33	.33	.44	.56

In 52 % of the outputs, the corrections were complete (CCC), meaning the feedback contained only correct corrections and suggestions, met the task requirements, and all issues of the student solution were addressed as part of the correction. In general, applying the corrections – even those with an incorrect explanation or reason – always resulted in a correct solution, except for two cases. In these two cases, the package was not corrected, but GPT-4 expressed that the package may need to be changed.

Student errors in the *SimpleWhileLoop* were related to the capitalization of the word “Boom!” and the loop. All of these capitalization errors were identified and corrected by the LLM. In a few cases, however, GPT-4 identified a “logical error” when the loop variable was not initialized with 1, loop-conditions were too complex, or unnecessary if-conditions were included in the student’s code.

A common student error in the *Queue* task was that the inner class for the *Node* was missing. This was always detected and corrected by GPT-4. Two student submissions had re-used an *ArrayList* or *LinkedList* (not singly-linked), which was also detected by GPT-4. GPT-4 also noted potential memory leaks (e. g., *tail* was not reset) in functional correct submissions and highlighted these as an error.

There are several cases in the *Queue* output where an error explanation is not correct, but the correction is. For example, GPT-4 correctly spotted an error in the *toArray* method. However, it reported an *ArrayIndexOutOfBoundsException*, whereas the actual error is a *NullPointerException*. In another example, a closing comment was not recognized as missing. GPT-4 also had problems in two runs with an inner class, which was named the same as the interface (Java allows such a scenario). Other issues occurred related to default initialized member variables. A missing explicit initialization in the constructor was often reported as an error. We also found problematic components of corrections (PCCS) such as “return an empty array not array with 0 length” or “The ‘remove’ method [...] doesn’t dispose of [sic] the removed object”. This is not correct for Java and may be misleading for students.

4.4 Suggested Optimizations and Coding Style

Most of the generated feedback texts (56 %) contained suggestions for optimizations (OPT, see Table 1). These can be characterized as either prescriptive *should do* and discretionary *could do*. Most optimizations relate to performance, such as incrementing a variable by two instead of two increments by one, or introducing a *tail* reference, and adding the field *size*. Further optimization suggestions (OPT) relate to discarding a (redundant) *size* field (with problematic time-complexity arguments), simplifying if-conditions, encapsulation, adding *@Override*, and avoiding to print errors to the console. None of the suggested prescriptive optimizations resulted in an

error or violation of the task specification. However, multiple aspects of the feedback may be too complex for students. This is particularly true for the *Queue* task, and concerns the handling of integer overflow, or taking care of concurrent access. Only one of the four provided concurrency fixes was correct. Moreover, we identified several inaccurate aspects, such as GPT suggesting to put the provided code for a private and/or static inner class into a separate file. In addition, some of the discretionary suggestions go beyond the task specification, as they recommend improving exception handling or using Generics.

Almost all of the generated feedback (96 %) contains code style suggestions (CSS). Most suggestions relate to the naming of variable/method/class (e. g., fixing typos, suggesting better names or CamelCase). Further suggestions encompass consistently using braces, proper indentation, using the interface constant instead of a literal, making variables final, avoiding redundant else blocks, deleting redundant comments, and only using *this* if necessary.

Regarding the language (LCS), GPT-4 provided feedback when comments or variable names were in German. The LLM suggested translating these to English, which we found in 18 % of the outputs.

4.5 Inconsistencies and Redundancies

Feedback inconsistencies (InC) are defined as recommendations that do not correspond to the sample solution or contradictions within the generated textual feedback. GPT-4 generated inconsistencies in about 22 % of the cases with a peak of 36 % in the third iteration of the *Queue*. A notable example for the *SimpleWhileLoop* is GPT-4 highlighting the initialization of *i* at 0 as a “Logical error” saying “that it should begin at 1”. At the same time, it acknowledged that starting at 0 (as in the submitted code) “would still produce the same correct sequence”. Similarly, for the *Queue*, the feedback presents an inconsistency: GPT-4 recommends not to use getters and setters for the inner class, but recommends marking “fields as private and accessing them through getters and setters”.

Redundancies (RD) were identified in about 10 % of the feedback outputs. For instance, some corrections or suggestions had already been part of the student’s submission. Feedback was also deemed redundant when it involved repeated or trivial suggestions, e. g., “‘QueueEntry’ should be named ‘QueueEntry’”.

5 DISCUSSION

The characterization of the feedback generated by GPT-4 shows that there are significant differences, compared to its previous version (GPT-3.5) and other LLMs. For example, the generated feedback is much longer, more structured, and got more complex. The median length is four times larger than reported in related work [1] (which used the same dataset). Studies on other models had reported issues with output formatting, such as an incorrect capitalization (cf. [1, 9]). We cannot confirm these issues for GPT-4. Furthermore, applying the suggestions or using the provided model solution of the feedback always leads to a completely correct solution (except for two cases, cf. subsection 4.3). Previous work had reported significant misleading information with GPT-3.5 [17]. Azaiz et al. showed an accuracy of the correctness classification of 73 %. Here, GPT-4 reaches 84 % on the same data set [1]. Azaiz et al. found significant differences in the feedback quality in response to fully

correct, syntactically incorrect, and functionally incorrect student submissions. This does not seem to be the case for GPT-4 Turbo. In this work, 52 % of the feedback was fully correct and complete, which only applied to 31 % of the outputs generated by GPT-3.5 in prior work [1]. Yet, 48 % of the generated feedback is not complete and fully correct w.r.t. all details. Overall, the generated feedback seems to be quite consistent across the three runs.

A benefit of using LLMs is that it generates 100 % personalized feedback without the need to develop test cases. This is a crucial advantage compared to traditional e-assessment systems that mostly provide simple informative feedback generated by test cases or compiler error messages [12]. The feedback by GPT-4 Turbo is more elaborated and always contained explanations and code. In contrast, GPT-3.5 not always offered code and text [1, 17, 18]. Another novelty compared to GPT-3.5 is that GPT-4 provides the output of students' code. Hints on possible memory leaks (not interfering with functional correctness), rejecting functional correct implementations not using a singly-linked list, variable naming, content of comments, performance optimizations, simplifications of code, and coding style show the potential of GPT-4 for using it as a tool for formative assessment. Before the broad availability of generative AI, detecting such issues required manual inspection, white-box testing, or professional tools.

As mentioned, the overall feedback is very detailed, long, and not always well ordered. 48 % of the generated feedback is incomplete and/or not fully correct, containing incorrect classifications, redundancies, inconsistencies, or problematic explanations. These aspects can make it more difficult for students to understand the feedback, increasing the cognitive load [41]. Similarly, the wording is not always appropriate for novices. Some comments in the feedback mention, for example, generics, concurrency, or improvements on the provided interface, which are likely to overwhelm novices who do not yet know these concepts.

The generated feedback almost always contains a model solution but rarely code snippets with gaps and instructions. Even if there is no consent among experts about feedback strategies (cf. [12]), this approach does not seem to encourage students to improve their submissions step-by-step. Another aspect worth mentioning is the absence of motivational statements, which is in contrast to related work [17] and, above all, to human tutors [38]. GPT-4 Turbo also ignored a student's question as part of a code comment. A human tutor would likely not have done that.

To conclude, using GPT-4 Turbo for automatically generating feedback does not seem to be advisable. The same applies to students using it without guidance or prior instruction. Nevertheless, it may be used to support teaching assistants, or advanced students who understand basic concepts and thus the provided feedback. In practice, (malicious) prompt injections must also be prevented.

Further research should focus on the pedagogical integration of the feedback, its consistency, how it can be tailored to the prior knowledge of the students, and how it can be linked to (the progress of) a specific course. The error classifications provided by GPT-4 may also help build a student model as part of an adaptive learning system. At the same time, the inherent dependency on OpenAI should be noted, if LLMs like GPT are used in educational settings. Sending student submissions to a third party may raise privacy

concerns. Hence, locally installed or offline LLMs are worth further consideration and research.

6 THREATS TO VALIDITY

A limiting factor of this work is that OpenAI's model is under active development. Therefore, we documented when the experiment was conducted, which model was used, and how the output was generated. However, it should be noted that GPT-4 Turbo is designed to predict subsequent tokens from previous ones. This is why its answers can vary upon regeneration, even when presented with identical prompts and inputs. For this reason, each submission was submitted to GPT-4 three times. The obtained results may have also been influenced by other factors, such as the programming language and the task specifications, which can vary across institutions. Finally, the limitations of the qualitative research paradigm and the content analysis technique are acknowledged. To ensure an intersubjective understanding and reliability, all three authors were involved in the classification.

7 CONCLUSIONS AND OUTLOOK

In the context of large introductory programming classes and educators' limited resources for providing individual feedback, Large Language Models may be helpful for learners to provide feedback, e. g., when they are stuck. Due to the well-known challenges of LLMs (e. g., falsifications, "lying"), however, it is crucial to evaluate its feedback characteristics before applying it in a course context with students or incorporating it into a learning system.

Therefore, the present work explored and characterized GPT-4 Turbo's output when prompted with an introductory programming task (*SimpleWhileLoop* and *Queue*) and respective student solutions, which were selected from a dataset gathered in an introductory programming course. The qualitative thematic analysis of the generated feedback texts revealed that all of the generated feedback is personalized. Moreover, the application of all corrections and suggestions in the feedback would have resulted in achieving the fully correct solution – except for two cases. However, only 52 % of the provided feedback was actually complete and fully correct in all details. In addition, the feedback provided actionable information on how to optimize the code and recommended stylistic changes for the majority of submissions.

We conclude that GPT-4 provides significantly improved feedback compared to older versions, as it performs better (e. g., accuracy). It correctly recognizes output formatting and provides more structured feedback. At the same time, there are still issues such as misleading feedback, incorrect/problematic explanations for corrections, redundancies, and inconsistencies within a generated feedback, but also across all outputs.

The present work offers several pathways for future work, such as the evaluation of the feedback from a pedagogical perspective, how well it addresses learner's informational needs, and how to integrate specific feedback categories into learning environments or formative assessment systems.

We thank all the students for their consent to use their submissions for (this) research. This research is part of the project AIM@LMU funded by the German Federal Ministry of Education and Research (BMBF) under the grant number 16DHBKI013.

REFERENCES

- [1] Imen Azaiz, Oliver Deckarm, and Sven Strickroth. 2023. AI-enhanced Auto-Correction of Programming Exercises: How Effective is GPT-3.5? *International Journal of Engineering Pedagogy (iJEP)* 13, 8 (Dec. 2023), 67–83. <https://doi.org/10.3991/ijep.v13i8.45621>
- [2] Brett A. Becker, Michelle Craig, Paul Denny, Hieke Keuning, Natalie Kiesler, Juho Leinonen, Andrew Luxton-Reilly, James Prather, and Keith Quille. 2023. Generative AI in Introductory Programming. (2023). <https://csed.acm.org/wp-content/uploads/2023/12/Generative-AI-Nov-2023-Version.pdf>
- [3] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, Dennis J. Bouvier, Brian Harrington, Amir Kamil, Amey Karkare, Chris McDonald, Peter-Michael Osera, Janice L. Pearce, and James Prather. 2019. Compiler Error Messages Considered Unhelpful. In *Proc. ITiCSE-WGR*. ACM. <https://doi.org/10.1145/3344429.3372508>
- [4] Douglas Bengtsson and Axel Kaliff. 2023. Assessment Accuracy of a Large Language Model on Programming Assignments. <https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-331000>
- [5] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. <https://doi.org/10.1191/147808706qp0630a>
- [6] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2023. Computing Education in the Era of Generative AI. *arXiv:2306.02608* [cs.CY] <https://doi.org/10.48550/arXiv.2306.02608>
- [7] Benedict Du Boulay. 1986. Some difficulties of learning to program. *Journal of Educational Computing Research* 2, 1 (1986), 57–73. <https://doi.org/10.2190/3LFX-9RRF-67T8-UVK9>
- [8] Michael Ebert and Markus Ring. 2016. A presentation framework for programming in programing lectures. In *Proc. EDUCON*. IEEE, 369–374.
- [9] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proc. ACE*. 10–19. <https://doi.org/10.1145/3511861.3511863>
- [10] James Finnie-Ansley, Paul Denny, Andrew Luxton-Reilly, Eddie Antonio Santos, James Prather, and Brett A. Becker. 2023. My AI Wants to Know if This Will Be on the Exam: Testing OpenAI’s Codex on CS2 Programming Exercises. In *Proc. ACE*. 97–104. <https://doi.org/10.1145/3576123.3576134>
- [11] Qiang Hao, David H Smith IV, Lu Ding, Amy Ko, Camille Ottaway, Jack Wilson, Kai H Arakawa, Alistair Turcan, Timothy Poehlman, and Tyler Greer. 2022. Towards understanding the effective design of automated formative feedback for programming assignments. *Computer Science Education* 32, 1 (2022), 105–127.
- [12] Johan Jeuring, Hieke Keuning, Samiha Marwan, Dennis Bouvier, Cruz Izu, Natalie Kiesler, Teemu Lehtinen, Dominic Lohr, Andrew Peterson, and Sami Sarsa. 2022. Towards Giving Timely Formative Feedback and Hints to Novice Programmers. In *Proceedings of the 2022 Working Group Reports on Innovation and Technology in Computer Science Education* (Dublin, Ireland) (ITiCSE-WGR ’22). ACM, New York, 95–115. <https://doi.org/10.1145/3571785.3574124>
- [13] Hieke Keuning, Bastiaan Heeren, and Johan Jeuring. 2021. A Tutoring System to Learn Code Refactoring. In *Proc. SIGCSE*. ACM. <https://doi.org/10.1145/3408877.3432526>
- [14] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *TOCE* 19, 1, Article 3 (2018). <https://doi.org/10.1145/3231711>
- [15] Natalie Kiesler. 2020. Towards a Competence Model for the Novice Programmer Using Bloom’s Revised Taxonomy – An Empirical Approach. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education* (Trondheim, Norway) (ITiCSE ’20). ACM, New York, 459–465. <https://doi.org/10.1145/3341525.3387419>
- [16] Natalie Kiesler. 2024. *Modeling Programming Competency : A Qualitative Analysis*. Springer International Publishing, Cham. 165 pages. <https://doi.org/10.1007/978-3-031-47148-3>
- [17] Natalie Kiesler, Dominic Lohr, and Hieke Keuning. 2024. Exploring the Potential of Large Language Models to Generate Formative Programming Feedback. In *2023 IEEE Frontiers in Education Conference (FIE)*. 1–5. <https://doi.org/10.1109/FIE58773.2023.10343457>
- [18] Natalie Kiesler and Daniel Schiffner. 2023. Large Language Models in Introductory Programming Education: ChatGPT’s Performance and Implications for Assessments. In *CoRR abs/2308.08572*. *arXiv: 2308.08572*. <https://doi.org/10.48550/arXiv.2308.08572> *arXiv:2308.08572* [cs.SE]
- [19] Nguyen-Thinh Le, Sven Strickroth, Sebastian Gross, and Niels Pinkwart. 2013. A Review of AI-Supported Tutoring Approaches for Learning Programming. In *Advanced Computational Methods for Knowledge Engineering - Proceedings of the 1st International Conference on Computer Science, Applied Mathematics and Applications (ICCSAMA) (Studies in Computational Intelligence, 479)*. Springer Verlag, Berlin, Germany, 267–279. https://doi.org/10.1007/978-3-319-00293-4_20
- [20] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing Code Explanations Created by Students and Large Language Models. In *Proc. ITiCSE*. 124–130. <https://doi.org/10.1145/3587102.3588785>
- [21] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A. Becker. 2023. Using Large Language Models to Enhance Programming Error Messages. In *Proc. SIGCSE*. ACM. <https://doi.org/10.1145/3545945.3569770>
- [22] Xiao Liu and Gyun Woo. 2020. Applying Code Quality Detection in Online Programming Judge. In *Proc. International Conference on Intelligent Information Technology (ICIIT 2020)*. ACM. <https://doi.org/10.1145/3385209.3385226>
- [23] Andrew Luxton-Reilly. 2016. Learning to Program is Easy. In *Proc. ITiCSE (ITiCSE ’16)*. 284–289. <https://doi.org/10.1145/2899415.2899432>
- [24] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory Programming: A Systematic Literature Review. In *Proc. ITiCSE*. ACM, New York, 55–106. <https://doi.org/10.1145/3293881.3295779>
- [25] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2023. Experiences From Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book. In *Proc. SIGCSE TS*. 931–937. <https://doi.org/10.1145/3545945.3569785>
- [26] Philipp Mayring. 2001. Combination and Integration of Qualitative and Quantitative Analysis. *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research* Vol 2 (2001). <https://doi.org/10.17169/FQS-2.1.967>
- [27] Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. *Handbook of research on educational communications and technology* 3 (2008), 125–144.
- [28] OpenAI. [n.d.]. GPT-4 Turbo. <https://help.openai.com/en/articles/8555510-gpt-4-turbo>
- [29] Andrew Petersen, Michelle Craig, Jennifer Campbell, and Anya Tafiiovich. 2016. Revisiting why students drop CS1. In *Proc. Koli Calling*. 71–80. <https://doi.org/10.1145/2999541.2999552>
- [30] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education* (Turku, Finland) (ITiCSE-WGR ’23). ACM, New York, 108–159. <https://doi.org/10.1145/3623762.3633499>
- [31] Lianne Roest, Hieke Keuning, and Johan Jeuring. 2023. Next-Step Hint Generation for Introductory Programming Using Large Language Models. *arXiv:2312.10055* [cs.CY]
- [32] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proc. ICER*. ACM. <https://doi.org/10.1145/3501385.3543957>
- [33] Jaromir Savelka, Arav Agarwal, Christopher Bogart, and Majd Sakr. 2023. Large Language Models (GPT) Struggle to Answer Multiple-Choice Questions about Code. *arXiv:2303.08033* [cs.CL]
- [34] Valerie J. Shute. 2008. Focus on formative feedback. *Review of Educational Research* 78, 1 (2008). <https://doi.org/10.3102/0034654307313795>
- [35] James C. Spohrer and Elliot Soloway. 1986. Novice mistakes: Are the folk wisdoms correct? *Communications of the ACM* 29, 7 (1986), 624–632. <https://doi.org/10.1145/6138.6145>
- [36] Sven Strickroth. 2023. Does Peer Code Review Change My Mind on My Submission?. In *Proc. ITiCSE*. 498–504. <https://doi.org/10.1145/3587102.3588802>
- [37] Sven Strickroth and François Bry. 2022. The Future of Higher Education is Social and Personalized! Experience Report and Perspectives. In *Proc. CSEDU*, Vol. 1. 389–396. <https://doi.org/10.5220/0011087700003182>
- [38] Sven Strickroth and Florian Holzinger. 2022. Supporting the Semi-Automatic Feedback Provisioning on Programming Assignments. In *Proc. MIS4TEL*. Springer International Publishing, Cham, 13–19. https://doi.org/10.1007/978-3-031-20617-7_3
- [39] Sven Strickroth, Hannes Olivier, and Niels Pinkwart. 2011. Das GATE-System: Qualitätssteigerung durch Selbsttests für Studenten bei der Onlineabgabe von Übungsaufgaben?. In *Proc. DeLFI*. Gesellschaft für Informatik e.V., Bonn, 115–126. <https://dl.gi.de/handle/20.500.12116/4740>
- [40] Sven Strickroth and Michael Striewe. 2022. Building a Corpus of Task-Based Grading and Feedback Systems for Learning and Teaching Programming. *International Journal of Engineering Pedagogy (iJEP)* 12, 5 (2022), 26–41. <https://doi.org/10.3991/ijep.v12i5.31283>
- [41] John Sweller. 1994. Cognitive load theory, learning difficulty, and instructional design. *Learning and instruction* 4, 4 (1994), 295–312. [https://doi.org/10.1016/0959-4752\(94\)90003-5](https://doi.org/10.1016/0959-4752(94)90003-5)
- [42] Jacqueline Whalley, Tony Clear, and Raymond Lister. 2007. The many ways of the Bracelet project. *BACIT* (2007).
- [43] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large Language Models in Fault Localisation. *arXiv:2308.15276* [cs.SE]